

Alberta Buck Identity - Cryptographic Example

Perry Kundert

2026-04-10

This document implements each cryptographic step of the Alberta Buck identity system: Pointcheval-Sanders signature issuance, credential derivation via the Identity Fountain, on-chain registration verification, ElGamal re-encryption with Chaum-Pedersen proof, and identity verification by a counterparty. All operations use the `alt_bn128` (BN254) curve – the same curve supported by Ethereum’s precompiles – via the `py_ecc` library.

We demonstrate a transfer from Private Alice to Public Bob: Alice holds a PS-signed identity, derives a fresh unlinkable credential, registers it, then re-encrypts her identity for Bob with a Chaum-Pedersen proof of correct re-encryption. Bob verifies Alice’s identity using the two-channel design: on-chain proof chain plus off-chain `identity_data`.

Contents

1	Setup: The <code>alt_bn128</code> Curve and Helpers	2
2	Issuer Key Generation	3
3	KYC Ceremony – Issuer Signs Alice’s Identity	3
3.1	Alice’s Plaintext Identity	3
3.2	Compute Identity Scalar and PS Signature	4
3.3	Verify the PS Signature (Issuer Self-Check)	4
4	The Identity Fountain – Alice Derives a Fresh Credential	4
4.1	Rerandomize the PS Signature	5
4.2	Generate Fresh Identity Key Pair and ElGamal Encryption	5
4.3	NIZK Proof: Binding PS Signature to ElGamal Ciphertext	6
5	Registration – On-Chain Verification	7
5.1	Verify the NIZK Proof (What the Contract Computes)	7
6	Bob Registers as a Public Account	9
7	Alice Approves Bob – Re-Encryption + Chaum-Pedersen Proof	10
7.1	Re-Encrypt Alice’s Identity for Bob	10
7.2	Chaum-Pedersen Proof of Correct Re-Encryption	10

7.3	Verify Chaum-Pedersen Proof (What the Contract Computes)	11
8	Transfer – Bilateral Identity Check	12
9	Bob Verifies Alice’s Identity (Two-Channel Design)	13
10	Receipt Construction	14
11	Summary: What Each Party Knows	14
12	Appendix: The Identity Fountain – Unlinkability Demonstration	15

1 Setup: The alt_bn128 Curve and Helpers

All operations use BN254, the curve behind Ethereum’s `ecAdd` (0x06), `ecMul` (0x07), and `ecPairing` (0x08) precompiles. The `py_ecc` library provides exactly these primitives.

```
import secrets
from py_ecc.bn128 import bn128_curve as bc, bn128_pairing as bp
from web3 import Web3

G1      = bc.G1          # Generator of G_1
G2      = bc.G2          # Generator of G_2
Z1      = bc.Z1          # Point at infinity (G_1 identity)
ORDER   = bc.curve_order # Curve group order (prime)
mul     = bc.multiply    # Scalar multiplication in G_1 or G_2
add     = bc.add         # Point addition
neg     = bc.neg         # Point negation
eq      = bc.eq          # Point equality
pairing = bp.pairing     # Optimal Ate pairing: G_2 x G_1 -> G_T

def rand():
    """Random non-zero scalar mod ORDER."""
    return secrets.randbelow(ORDER - 1) + 1

def keccak(*args):
    """keccak256 over concatenated big-endian 32-byte words, reduced mod ORDER."""
    data = b''.join(v.to_bytes(32, 'big') if isinstance(v, int)
                    else str(v).encode() for v in args)
    h = Web3.solidity_keccak(['bytes'], [data])
    return int.from_bytes(h, 'big') % ORDER

def pt(P, label=""):
    """Format a G_1 point for display."""
    if bc.is_inf(P):
        return f"{label}0 (infinity)"
    return f"{label}({int(P[0]) % 10**6:>6d}..., {int(P[1]) % 10**6:>6d}...)"

print(f"BN254 curve order: {ORDER}")
print(f"G_1 generator:      {pt(G1)}")
print(f"G_2 generator:      (pair of FQ2 elements)")

BN254 curve order: 21888242871839275222246405745257275088548364400416034343698204186575808495617
G_1 generator:      ( 1..., 2...)
G_2 generator:      (pair of FQ2 elements)
```

2 Issuer Key Generation

The identity issuer (e.g., ATB Financial acting as Alberta’s KYC authority) generates a Pointcheval-Sanders key pair. The secret key is two scalars (x, y) ; the public key is two points in G_2 .

```
# Issuer's PS secret key
isk_x = rand()
isk_y = rand()

# Issuer's PS public key (in G_2, published on-chain in TrustedIssuersRegistry)
IPK_X = mul(G2, isk_x)
IPK_Y = mul(G2, isk_y)

print("Issuer PS key pair generated.")
print(f"  sk = (x, y)           -- two secret scalars")
print(f"  PK = (X, Y) in G_2      -- published in TrustedIssuersRegistry")

Issuer PS key pair generated.
  sk = (x, y)           -- two secret scalars
  PK = (X, Y) in G_2      -- published in TrustedIssuersRegistry
```

3 KYC Ceremony – Issuer Signs Alice’s Identity

Alice presents her identity documents. The issuer verifies them, constructs a canonical `identity_data` structure, computes the identity scalar $m = H(\text{identity_data})$, and produces a PS signature.

3.1 Alice’s Plaintext Identity

```
import json

# Alice's verified identity (canonical encoding -- field order matters)
alice_identity_data = json.dumps({
    "given_name":    "Alice",
    "family_name":   "Johnson",
    "jurisdiction":  "Alberta, Canada",
    "id_type":       "Alberta Identity Card",
    "id_number":     "AIC-2026-4839201",
    "date_of_birth": "1992-03-15",
    "issuer_id":     "atb-financial-ca",
    "issued_at":     "2026-01-20T14:30:00Z",
    "epoch":         42
}, sort_keys=True, separators=(',', ':'))

print("Alice's identity_data (canonical JSON):")
print(f"  {alice_identity_data[:72]}...")

Alice's identity_data (canonical JSON):
{"date_of_birth":"1992-03-15","epoch":42,"family_name":"Johnson","given_...
```

3.2 Compute Identity Scalar and PS Signature

The PS signature is $\sigma = (h, (x + m \cdot y) \cdot h)$ where h is a random point in G_1 .

```
# Identity scalar: m = keccak256(identity_data) mod ORDER
m_alice = keccak(alice_identity_data)

# PS signature: pick random h, compute sigma = (h, (x + m*y) * h)
t_h = rand()
h = mul(G1, t_h) # random G_1 point
sigma_2 = mul(h, (isk_x + m_alice * isk_y) % ORDER)
sigma = (h, sigma_2)

print(f"Identity scalar m = H(identity_data):")
print(f"  m = {m_alice}")
print(f"PS signature sigma:")
print(f"  sigma_1 = {pt(sigma[0])}")
print(f"  sigma_2 = {pt(sigma[1])}")

Identity scalar m = H(identity_data):
  m = 6446313962173165014188361468977035479170665192834402473818305319846425860127
PS signature sigma:
  sigma_1 = (852494..., 421539...)
  sigma_2 = (273128..., 321266...)
```

3.3 Verify the PS Signature (Issuer Self-Check)

The PS verification equation uses a bilinear pairing:

$$e(\sigma_1, X + m \cdot Y) = e(\sigma_2, g_2)$$

```
# Verify: e(sigma_1, X + m*Y) == e(sigma_2, g_2)
lhs = pairing(add(IPK_X, mul(IPK_Y, m_alice)), sigma[0])
rhs = pairing(G2, sigma[1])
assert lhs == rhs, "PS signature verification FAILED"
print("PS signature verified: e(sigma_1, X + m*Y) == e(sigma_2, g_2)")

# The issuer gives Alice: (m, sigma, identity_data)
# Alice stores all three in her Holochain Private entry
print(f"\nIssuer gives Alice:")
print(f"  m = {m_alice}")
print(f"  sigma = (sigma_1, sigma_2) -- PS signature")
print(f"  identity_data = '{alice_identity_data[:40]}...'")

PS signature verified: e(sigma_1, X + m*Y) == e(sigma_2, g_2)

Issuer gives Alice:
  m = 6446313962173165014188361468977035479170665192834402473818305319846425860127
  sigma = (sigma_1, sigma_2) -- PS signature
  identity_data = '{"date_of_birth":"1992-03-15","epoch":42...'
```

4 The Identity Fountain – Alice Derives a Fresh Credential

Alice wants to create a new Ethereum account. Her Holochain wallet derives a fresh, unlinkable credential: a rerandomized PS signature, a new key pair, an ElGamal encryption of m , and a NIZK proof binding them. This runs entirely offline on Alice’s device.

4.1 Rerandomize the PS Signature

Alice picks a random scalar t and computes $\sigma' = (t \cdot \sigma_1, t \cdot \sigma_2)$. This is a valid PS signature on the same m – but *statistically unlinkable* to the original σ .

```
# Rerandomize: sigma' = (t * sigma_1, t * sigma_2)
t_rerand = rand()
sigma_p_1 = mul(sigma[0], t_rerand)
sigma_p_2 = mul(sigma[1], t_rerand)
sigma_p = (sigma_p_1, sigma_p_2)

# Verify sigma' is still a valid PS signature on m
lhs = pairing(add(IPK_X, mul(IPK_Y, m_alice)), sigma_p[0])
rhs = pairing(G2, sigma_p[1])
assert lhs == rhs, "Rerandomized PS signature FAILED"
assert not bc.is_inf(sigma_p[0]), "sigma'_1 must not be point at infinity"
print("Rerandomized PS signature verified.")
print(f"  sigma'_1 = {pt(sigma_p[0])}")
print(f"  sigma'_2 = {pt(sigma_p[1])}")
print(f"\nOriginal and rerandomized signatures are unlinkable:")
print(f"  sigma_1 = {pt(sigma[0])}")
print(f"  sigma'_1 = {pt(sigma_p[0])}")
print(f"  (No algorithm can correlate them.)")
```

```
Rerandomized PS signature verified.
  sigma'_1 = (469693..., 489539...)
  sigma'_2 = (761896..., 547654...)

Original and rerandomized signatures are unlinkable:
  sigma_1 = (852494..., 421539...)
  sigma'_1 = (469693..., 489539...)
  (No algorithm can correlate them.)
```

4.2 Generate Fresh Identity Key Pair and ElGamal Encryption

Alice generates a new key pair on `alt_bn128` and encrypts her identity scalar m under her own public key using ElGamal:

$$E_{\text{alice}} = (r \cdot G, m \cdot G + r \cdot pk_{\text{alice}})$$

```
# Fresh identity key pair for this account
sk_alice = rand()
pk_alice = mul(G1, sk_alice)

# ElGamal encryption of m under pk_alice
r_enc = rand()
R_alice = mul(G1, r_enc)
M_alice = mul(G1, m_alice)
C_alice = add(M_alice, mul(pk_alice, r_enc))
E_alice = (R_alice, C_alice)

# R = r * G
# M = m * G (message point)
# C = m*G + r*pk

print(f"Fresh key pair for this Ethereum account:")
print(f"  sk_alice = {sk_alice}")
print(f"  pk_alice = {pt(pk_alice)}")
print(f"\nElGamal ciphertext E_alice = (R, C):")
print(f"  R = r*G = {pt(R_alice)}")
print(f"  C = m*G+r*pk = {pt(C_alice)}")
print(f"\nMessage point M = m*G:")
```

```

print(f" M = {pt(M_alice)}")

# Verify Alice can decrypt her own ciphertext
M_check = add(C_alice, neg(mul(R_alice, sk_alice))) # C - sk*R = M
assert eq(M_check, M_alice), "ElGamal self-decryption FAILED"
print(f"\nAlice decrypts: C - sk*R = {pt(M_check)} (matches M)")

Fresh key pair for this Ethereum account:
sk_alice = 1696140636709269414761873094975974951671351664858046710223266985216790365077
pk_alice = (199337..., 148784...)

ElGamal ciphertext E_alice = (R, C):
R = r*G = (540780..., 767503...)
C = m*G+r*pk = (871046..., 790930...)

Message point M = m*G:
M = (739132..., 937433...)

Alice decrypts: C - sk*R = (739132..., 937433...) (matches M)

```

4.3 NIZK Proof: Binding PS Signature to ElGamal Ciphertext

Alice must prove (without revealing m or r) that:

- (a) σ' is a valid PS signature on m
- (b) E_{alice} encrypts the same m

This is a Schnorr-family sigma protocol. The PS relation is *linear* in m , so no Groth-Sahai proofs or SNARKs are needed.

```

# === PROVER (Alice, offline) ===
# Commit: pick random blinding scalars
m_tilde = rand() # blinding for m
r_tilde = rand() # blinding for r

# Commitment for PS equation (a): A_ps = m_tilde * sigma'_1
# (This will be checked against the pairing equation)
A_ps = mul(sigma_p[0], m_tilde)

# Commitments for ElGamal equations (b) and (c):
T_C = add(mul(G1, m_tilde), mul(pk_alice, r_tilde)) # m~*G + r~*pk
T_R = mul(G1, r_tilde) # r~*G

# Fiat-Shamir challenge: hash all public inputs + commitments
# (In Ethereum: includes registrant_address for domain separation)
e = keccak(
    sigma_p[0][0], sigma_p[0][1], # sigma'_1
    sigma_p[1][0], sigma_p[1][1], # sigma'_2
    R_alice[0], R_alice[1], # E_alice.R
    C_alice[0], C_alice[1], # E_alice.C
    pk_alice[0], pk_alice[1], # pk_alice
    A_ps[0], A_ps[1], # PS commitment
    T_C[0], T_C[1], # ElGamal C commitment
    T_R[0], T_R[1], # ElGamal R commitment
)

# Responses

```

```

s_m = (m_tilde + e * m_alice) % ORDER
s_r = (r_tilde + e * r_enc) % ORDER

pi = (e, s_m, s_r, A_ps, T_C, T_R)
print("NIZK proof pi = (e, s_m, s_r, A_ps, T_C, T_R)")
print(f" challenge e = {e}")
print(f" response s_m = {s_m}")
print(f" response s_r = {s_r}")

NIZK proof pi = (e, s_m, s_r, A_ps, T_C, T_R)
challenge e = 4898639945517814188497184278382597778878973591318190984973799168036015039325
response s_m = 10593781547917235693061492950234347756280855093294798310952287176635503601924
response s_r = 18150976040332756963959706717327320343601112622393902018306749786312716914064

```

5 Registration – On-Chain Verification

Alice submits $(pk, E_{\text{alice}}, \sigma', \pi)$ to the IdentityRegistry. The contract verifies the PS signature and NIZK proof using `ecPairing`, `ecMul`, and `ecAdd` precompiles (~235K gas).

5.1 Verify the NIZK Proof (What the Contract Computes)

```

# === VERIFIER (IdentityRegistry contract, on-chain) ===
# Unpack proof
e_v, s_m_v, s_r_v, A_ps_v, T_C_v, T_R_v = pi

# --- Check (b): ElGamal consistency ---
# Verify: s_m * G + s_r * pk_alice == e * C + T_C
lhs_C = add(mul(G1, s_m_v), mul(pk_alice, s_r_v))
rhs_C = add(mul(C_alice, e_v), T_C_v)
assert eq(lhs_C, rhs_C), "ElGamal C check FAILED"
print("(b) ElGamal C check passed: s_m * G + s_r * pk == e * C + T_C")

# Verify: s_r * G == e * R + T_R
lhs_R = mul(G1, s_r_v)
rhs_R = add(mul(R_alice, e_v), T_R_v)
assert eq(lhs_R, rhs_R), "ElGamal R check FAILED"
print("(c) ElGamal R check passed: s_r * G == e * R + T_R")

# --- Check (a): PS signature via pairing ---
# Reconstruct PS commitment: s_m * sigma'_1 should equal A_ps + e * sigma'_2
# relative to the pairing equation.
# Verify pairing product:
#   e(s_m * sigma'_1, Y) * e(-A_ps, Y) * e(e * sigma'_1, X)
#   * e(-e * sigma'_2, g_2) == 1
# Which simplifies to:
#   e(s_m * sigma'_1 - A_ps, Y) * e(e * sigma'_1, X) * e(-e * sigma'_2, g_2) == 1
#
# This is equivalent to checking:
#   e(A_ps, Y) * e(e * sigma'_2, g_2) == e(s_m * sigma'_1, Y) * e(e * sigma'_1, X)

P1 = mul(sigma_p[0], s_m_v) # s_m * sigma'_1
P2 = add(A_ps_v, mul(sigma_p[1], e_v)) # A_ps + e * sigma'_2
P3 = mul(sigma_p[0], e_v) # e * sigma'_1

# Pairing check: e(P1, Y) == e(P2, Y) * e(P3, X) / e(P3, X)
# More directly: verify the original PS equation holds through the proof
# e(s_m * sigma'_1 - A_ps, Y) == e(A_ps + e * sigma'_1, X) ... etc.
#

```

```

# Simplified: check that the reconstructed pairing equation holds:
#  $e(\sigma'_1, X + mY) = e(\sigma'_2, g_2)$  via the proof responses
#
# The verifier checks:  $e(s_m s'_1, Y) * e(e s'_1, X) == e(A_{ps}, Y) * e(e s'_2, g_2)$ 
lhs_pair = pairing(IPK_Y, P1) * pairing(IPK_X, P3)
rhs_pair = pairing(IPK_Y, P2) * pairing(G2, mul(sigma_p[1], e_v))
# Actually let's use the direct verification approach:
# We know  $s_m = m + e m$ . The commitment  $A_{ps} = m \sigma'_1$ .
# So  $s_m \sigma'_1 = m \sigma'_1 + e m \sigma'_1 = A_{ps} + e m \sigma'_1$ 
# The verifier doesn't know  $m$ , but can check via pairing:
#  $e(s_m \sigma'_1, Y) * e(e \sigma'_1, X) * e(-e \sigma'_2 - A_{ps}, g_2\_placeholder)$ 
# This gets complex in code. Let's use the direct pairing product check.

# Direct approach: verify  $e(\sigma'_1, X + mY) = e(\sigma'_2, g_2)$ 
# by extracting  $m$  from the sigma protocol responses:
# The key insight: the SAME  $s_m$  that satisfies (b) must also satisfy (a).
# If checks (b) and (c) pass, we know  $s_m = m + e m$  for the  $m$  in  $E_{alice}$ .
# Now check (a): does this same  $m$  satisfy the PS equation?
#
# Reconstruct:  $m * \sigma'_1 = (s_m * \sigma'_1 - A_{ps}) / e$ 
# But we avoid division. Instead verify the pairing product equation:
#  $e(s_m * \sigma'_1 - A_{ps}, Y) == e(e * \sigma'_2, g_2) * e(-e * \sigma'_1, X)$ 
# (rearranging  $e(\sigma'_1, X + mY) = e(\sigma'_2, g_2)$  with substitution)

LHS_point = add(mul(sigma_p[0], s_m_v), neg(A_ps_v)) #  $s_m s'_1 - A_{ps} = e m s'_1$ 
RHS_neg = add(mul(sigma_p[1], e_v), neg(mul(sigma_p[0], e_v))) #  $e s'_2 - e s'_1$ 
# not quite right -- let me use the clean verification

# Clean pairing product check (what ecPairing precompile computes):
#  $e(s_m \sigma'_1, Y) * e(-A_{ps}, Y) * e(e \sigma'_1, X) * e(-e \sigma'_2, g_2) == 1$ 
check = (
    pairing(IPK_Y, mul(sigma_p[0], s_m_v)) #  $e(s_m s'_1, Y)$ 
    * pairing(IPK_Y, neg(A_ps_v)) #  $e(-A_{ps}, Y)$ 
    * pairing(IPK_X, mul(sigma_p[0], e_v)) #  $e(e s'_1, X)$ 
    * pairing(G2, neg(mul(sigma_p[1], e_v))) #  $e(-e s'_2, g_2)$ 
)
assert check == bp.FQ12.one(), "PS pairing check FAILED"
print("(a) PS pairing check passed:")
print("     $e(s_m s'_1, Y) * e(-A_{ps}, Y) * e(e s'_1, X) * e(-e s'_2, g_2) == 1$ ")

# --- Fiat-Shamir challenge reconstruction ---
e_reconstructed = keccak(
    sigma_p[0][0], sigma_p[0][1],
    sigma_p[1][0], sigma_p[1][1],
    R_alice[0], R_alice[1],
    C_alice[0], C_alice[1],
    pk_alice[0], pk_alice[1],
    A_ps_v[0], A_ps_v[1],
    T_C_v[0], T_C_v[1],
    T_R_v[0], T_R_v[1],
)
assert e_reconstructed == e_v, "Fiat-Shamir challenge mismatch"
print("(d) Fiat-Shamir challenge verified (binds all public inputs)")

# ---  $\sigma'_1 \neq 0$  check ---
assert not bc.is_inf(sigma_p[0]), "sigma'_1 is point at infinity (trivial forgery)"
print("(e)  $\sigma'_1 \neq 0$  (non-trivial signature)")

print("\n=> Registration PASSED. isVerified(alice) = true")

(b) ElGamal C check passed:  $s_m G + s_r pk == e C + T_C$ 
(c) ElGamal R check passed:  $s_r G == e R + T_R$ 

```



```

(a) PS pairing check passed:
    e(s_m*s'_1, Y) * e(-A_ps, Y) * e(e*s'_1, X) * e(-e*s'_2, g_2) == 1
(d) Fiat-Shamir challenge verified (binds all public inputs)
(e) sigma'_1 != 0 (non-trivial signature)

==> Registration PASSED. isVerified(alice) = true

```

6 Bob Registers as a Public Account

Bob is a Uniswap pool operator. He registers publicly: his `identity_data` is world-readable in the IdentityRegistry. This exempts his counterparties from needing Bob to call `approve`.

```

# Bob's identity (public pool operator)
bob_identity_data = json.dumps({
    "given_name": "Bob",
    "family_name": "Smith",
    "jurisdiction": "Alberta, Canada",
    "id_type": "Corporate Registration",
    "id_number": "AB-CORP-2026-00182",
    "date_of_birth": "1985-07-22",
    "issuer_id": "atb-financial-ca",
    "issued_at": "2026-02-01T09:00:00Z",
    "epoch": 42
}, sort_keys=True, separators=(',', ':'))

m_bob = keccak(bob_identity_data)

# Bob also gets a PS signature (same issuance ceremony)
t_h_bob = rand()
h_bob = mul(G1, t_h_bob)
sigma_bob = (h_bob, mul(h_bob, (isk_x + m_bob * isk_y) % ORDER))

# Bob generates key pair and ElGamal encryption
sk_bob = rand()
pk_bob = mul(G1, sk_bob)
r_bob = rand()
R_bob = mul(G1, r_bob)
M_bob = mul(G1, m_bob)
C_bob = add(M_bob, mul(pk_bob, r_bob))
E_bob_reg = (R_bob, C_bob)

# Bob registers (same PS+NIZK verification as Alice -- omitted for brevity)
# AND publishes identity_data on-chain (public account)
print("Bob registered as PUBLIC account:")
print(f" pk_bob = {pt(pk_bob)}")
print(f" E_bob = (R, C) -- ElGamal ciphertext on-chain")
print(f" identity_data = '{bob_identity_data[:50]}...'")
print(f" isVerified(bob) = true")
print(f" isPublic(bob) = true")
print(f"\nBob's identity is world-readable. No approve() needed from Bob's side.")

Bob registered as PUBLIC account:
pk_bob = (292298..., 451792...)
E_bob = (R, C) -- ElGamal ciphertext on-chain
identity_data = '{"date_of_birth":"1985-07-22","epoch":42,"family_n...'
isVerified(bob) = true
isPublic(bob) = true

Bob's identity is world-readable. No approve() needed from Bob's side.

```

7 Alice Approves Bob – Re-Encryption + Chaum-Pedersen Proof

Alice wants to swap BUCK for USDT on Bob’s pool. She must call `approve(pool, amount, E_bob, proof)` to re-encrypt her identity for Bob and prove the re-encryption is correct.

Since Bob is public, only Alice needs to approve. The bilateral identity check in transfer will pass: `isPublic(bob) = true` = satisfies Bob’s side.

7.1 Re-Encrypt Alice’s Identity for Bob

Alice decrypts her own ciphertext to recover $M = m \cdot G$, then re-encrypts under Bob’s public key with fresh randomness.

```
# Alice decrypts her own credential
M_decrypted = add(C_alice, neg(mul(R_alice, sk_alice))) # C - sk*R = M
assert eq(M_decrypted, M_alice), "Self-decryption failed"

# Re-encrypt M under Bob's public key with fresh randomness r'
r_prime = rand()
R_for_bob = mul(G1, r_prime) # R' = r' * G
C_for_bob = add(M_alice, mul(pk_bob, r_prime)) # C' = M + r' * pk_bob
E_for_bob = (R_for_bob, C_for_bob)

print("Alice re-encrypts her identity for Bob:")
print(f" E_bob = (R', C') where:")
print(f" R' = r'*G = {pt(R_for_bob)}")
print(f" C' = M+r'*pk = {pt(C_for_bob)}")

# Verify Bob can decrypt
M_bob_decrypts = add(C_for_bob, neg(mul(R_for_bob, sk_bob)))
assert eq(M_bob_decrypts, M_alice), "Bob cannot decrypt E_bob"
print(f"\nBob decrypts: C' - sk_bob*R' = {pt(M_bob_decrypts)}")
print(f"Matches Alice's M: {pt(M_alice)}")

Alice re-encrypts her identity for Bob:
E_bob = (R', C') where:
R' = r'*G = (281204..., 226277...)
C' = M+r'*pk = (726349..., 33575...)

Bob decrypts: C' - sk_bob*R' = (739132..., 937433...)
Matches Alice's M: (739132..., 937433...)
```

7.2 Chaum-Pedersen Proof of Correct Re-Encryption

Alice proves that E_{bob} encrypts the same message point M as her registered E_{alice} – without revealing m or any private key.

The proof demonstrates knowledge of the re-encryption randomness r' and the difference between the two ElGamal randomness values, such that both ciphertexts decrypt to the same M .

```
# Chaum-Pedersen proof that E_alice and E_bob encrypt the same M.
#
```

```

# The key relation: C_alice - sk_alice * R_alice = M = C_bob - r' * pk_bob
# Equivalently: C_alice - sk_alice * R_alice = C_bob - r' * pk_bob
#
# Alice proves she knows (sk_alice, r') such that:
#   sk_alice * R_alice = C_alice - M   (she can decrypt E_alice)
#   r' * pk_bob       = C_bob - M     (she created E_bob correctly)
#   r' * G             = R_bob        (consistent randomness)

# Commit: random blinders
k1 = rand() # blinder for sk_alice
k2 = rand() # blinder for r'

T1 = mul(R_alice, k1) # k1 * R_alice
T2 = mul(pk_bob, k2)  # k2 * pk_bob
T3 = mul(G1, k2)      # k2 * G

# Fiat-Shamir challenge (binds all public inputs + Ethereum context)
e_cp = keccak(
    R_alice[0], R_alice[1], C_alice[0], C_alice[1], # E_alice
    R_for_bob[0], R_for_bob[1], C_for_bob[0], C_for_bob[1], # E_bob
    pk_alice[0], pk_alice[1], # pk_alice
    pk_bob[0], pk_bob[1], # pk_bob
    T1[0], T1[1], T2[0], T2[1], T3[0], T3[1], # commitments
    42, 1337, # msg.sender, spender (stand-ins)
)

# Responses
s1_cp = (k1 + e_cp * sk_alice) % ORDER
s2_cp = (k2 + e_cp * r_prime) % ORDER

proof_cp = (e_cp, s1_cp, s2_cp)
print("Chaum-Pedersen proof:")
print(f" e = {e_cp}")
print(f" s1 = {s1_cp}")
print(f" s2 = {s2_cp}")

Chaum-Pedersen proof:
e = 3040091126131313156041857722759504420266691800191549430584304453809072245032
s1 = 821287529080520234004468399363324387452265918331132872273800797291747316479
s2 = 4695087169344847173043223073063214830348711693396458598433292548950840216820

```

7.3 Verify Chaum-Pedersen Proof (What the Contract Computes)

The BUCK contract reads E_{alice} from the IdentityRegistry (never from calldata), then verifies the proof. Cost: ~29,000 gas.

```

# === VERIFIER (BUCK contract, on-chain, ~29K gas) ===
e_v, s1_v, s2_v = proof_cp

# The verifier knows: E_alice (from registry), E_bob (from calldata),
# pk_alice (from registry), pk_bob (from registry).

# Reconstruct commitments from responses and challenge:
#   T1' = s1 * R_alice - e * (C_alice - C_bob + r'*pk_bob - sk*R_alice)
# Simplified verification equations:
#   s1 * R_alice == T1 + e * sk_alice * R_alice
#   == T1 + e * (C_alice - M)
# But verifier doesn't know M. Use the relation C_alice - sk*R = C_bob - r'*pk_bob = M:
#   s1 * R_alice - s2 * pk_bob == T1 - T2 + e*(C_alice - C_bob)
# And: s2 * G == T3 + e * R_bob

```

```

# Check 1:  $s_2 * G == T_3 + e * R_{\text{for\_bob}}$ 
lhs1 = mul(G1, s2_v)
rhs1 = add(T3, mul(R_for_bob, e_v))
assert eq(lhs1, rhs1), "Chaum-Pedersen check 1 FAILED"
print("Check 1 passed:  $s_2 * G == T_3 + e * R_{\text{bob}}$ ")
print(" (Proves r' is consistent with R_bob)")

# Check 2:  $s_1 * R_{\text{alice}} - s_2 * pk_{\text{bob}} == (T_1 - T_2) + e * (C_{\text{alice}} - C_{\text{bob}})$ 
lhs2 = add(mul(R_alice, s1_v), neg(mul(pk_bob, s2_v)))
rhs2 = add(add(T1, neg(T2)), mul(add(C_alice, neg(C_for_bob)), e_v))
assert eq(lhs2, rhs2), "Chaum-Pedersen check 2 FAILED"
print("Check 2 passed:  $s_1 * R_a - s_2 * pk_b == (T_1 - T_2) + e * (C_a - C_b)$ ")
print(" (Proves both ciphertexts encrypt the same M)")

# Check 3: Fiat-Shamir challenge
e_recon = keccak(
    R_alice[0], R_alice[1], C_alice[0], C_alice[1],
    R_for_bob[0], R_for_bob[1], C_for_bob[0], C_for_bob[1],
    pk_alice[0], pk_alice[1],
    pk_bob[0], pk_bob[1],
    T1[0], T1[1], T2[0], T2[1], T3[0], T3[1],
    42, 1337,
)
assert e_recon == e_v, "Fiat-Shamir challenge mismatch"
print("Check 3 passed: Fiat-Shamir challenge verified")

print("\n==> approve() PASSED. Alice's re-encrypted identity stored for Bob.")
print(" _receiptFragments[alice][bob] = keccak256(E_bob)")

Check 1 passed:  $s_2 * G == T_3 + e * R_{\text{bob}}$ 
(Proves r' is consistent with R_bob)
Check 2 passed:  $s_1 * R_a - s_2 * pk_b == (T_1 - T_2) + e * (C_a - C_b)$ 
(Proves both ciphertexts encrypt the same M)
Check 3 passed: Fiat-Shamir challenge verified

==> approve() PASSED. Alice's re-encrypted identity stored for Bob.
_receiptFragments[alice][bob] = keccak256(E_bob)

```

8 Transfer – Bilateral Identity Check

With Alice's `approve` done and Bob being a public account, the `transfer` can proceed. The contract checks that both parties' identities are available to each other.

```

# Simulating the contract's bilateral identity check:
is_public_alice = False
is_public_bob   = True
receipt_alice_to_bob = True # set during approve() above
receipt_bob_to_alice = False # Bob never called approve (he's public)

# Sender (Alice) identity available to receiver (Bob)?
sender_ok = is_public_alice or receipt_alice_to_bob
# Receiver (Bob) identity available to sender (Alice)?
receiver_ok = is_public_bob or receipt_bob_to_alice

print("Bilateral identity check:")
print(f" Alice (private): isPublic={is_public_alice}, "
      f"receiptFragment={receipt_alice_to_bob} => {sender_ok}")
print(f" Bob (public): isPublic={is_public_bob}, "

```

```

    f"receiptFragment={receipt_bob_to_alice} => {receiver_ok}")
print(f"\n sender_ok AND receiver_ok = {sender_ok and receiver_ok}")
print(f"\n==> transfer(bob, 500 BUCK) SUCCEEDS")
print(f"    BuckTransferReceipt emitted with identity hashes")

```

Bilateral identity check:

```

    Alice (private): isPublic=False, receiptFragment=True => True
    Bob   (public):  isPublic=True, receiptFragment=False => True

```

```

    sender_ok AND receiver_ok = True

```

```

==> transfer(bob, 500 BUCK) SUCCEEDS
    BuckTransferReceipt emitted with identity hashes

```

9 Bob Verifies Alice's Identity (Two-Channel Design)

Bob decrypts the ElGamal ciphertext to get the message point M , then receives Alice's `identity_data` via Holochain and verifies the binding: $H(\text{identity_data}) \cdot G = M$.

=== Bob's side ===

```

# Channel 1 (Ethereum proof chain): Bob decrypts E_bob
M_from_bob = add(C_for_bob, neg(mul(R_for_bob, sk_bob)))
print("Bob decrypts E_bob from on-chain IdentityExchange event:")
print(f"  M = C' - sk_bob * R' = {pt(M_from_bob)}")

# Channel 2 (Holochain data channel): Bob receives identity_data
received_identity_data = alice_identity_data # <-- via Holochain, encrypted for Bob
print(f"\nBob receives identity_data via Holochain:")
print(f"  '{received_identity_data[:60]}...'")

# Verify binding: H(identity_data) * G == M
m_check = keccak(received_identity_data)
M_check = mul(G1, m_check)
assert eq(M_check, M_from_bob), "Identity binding FAILED"

print(f"\nTwo-channel verification:")
print(f"  m = H(identity_data) = {m_check}")
print(f"  m * G = {pt(M_check)}")
print(f"  M      = {pt(M_from_bob)}")
print(f"  H(identity_data) * G == M : True")
print(f"\n==> Bob has cryptographic proof that the identity_data")
print(f"    he received is the same identity the issuer certified")
print(f"    and Alice registered on-chain.")

```

Bob decrypts E_bob from on-chain IdentityExchange event:

```

    M = C' - sk_bob * R' = (739132..., 937433...)

```

Bob receives identity_data via Holochain:

```

    '{"date_of_birth":"1992-03-15","epoch":42,"family_name":"John...'}

```

Two-channel verification:

```

    m = H(identity_data) = 6446313962173165014188361468977035479170665192834402473818305319846425860127
    m * G = (739132..., 937433...)
    M      = (739132..., 937433...)
    H(identity_data) * G == M : True

```

```

==> Bob has cryptographic proof that the identity_data
    he received is the same identity the issuer certified
    and Alice registered on-chain.

```

10 Receipt Construction

Both parties hold each other's plaintext `identity_data` (Alice has Bob's from the public IdentityRegistry; Bob has Alice's from Holochain). Both hold the transfer data from on-chain events. Either can independently construct the canonical receipt.

```
# Both parties construct the same canonical receipt
def canonical_receipt(sender_id, receiver_id, amount, tx_hash, block):
    return json.dumps({
        "sender": json.loads(sender_id),
        "receiver": json.loads(receiver_id),
        "amount": amount,
        "tx_hash": tx_hash,
        "block": block,
    }, sort_keys=True, separators=(',', ':'))

# Stand-in transaction data (would come from on-chain Transfer event)
tx_hash = "0xabcd1234567890"
block = 19400000
amount = 500

# Alice constructs her copy
receipt_alice = canonical_receipt(
    alice_identity_data, bob_identity_data, amount, tx_hash, block)

# Bob constructs his copy
receipt_bob = canonical_receipt(
    alice_identity_data, bob_identity_data, amount, tx_hash, block)

h_alice = keccak(receipt_alice)
h_bob = keccak(receipt_bob)

print("Receipt construction (independent, canonical):")
print(f" Alice's H(receipt) = {h_alice}")
print(f" Bob's H(receipt) = {h_bob}")
print(f" Match: {h_alice == h_bob}")
print(f"\nReceipt stored (encrypted) on each party's Holochain source chain.")
print(f"The receipt hash anchors the off-chain record to the on-chain transfer.")

Receipt construction (independent, canonical):
Alice's H(receipt) = 21535858141339728776948445993719093154923735092821067258494283187515610795617
Bob's H(receipt) = 21535858141339728776948445993719093154923735092821067258494283187515610795617
Match: True

Receipt stored (encrypted) on each party's Holochain source chain.
The receipt hash anchors the off-chain record to the on-chain transfer.
```

11 Summary: What Each Party Knows

```
[
    ["Artifact", "Alice", "Bob", "Eve", "Issuer"],
    None,
    ["identity_data (Alice)", "YES (owns it)", "YES (Holochain)", "NO", "YES (issued it)"],
    ["identity_data (Bob)", "YES (public)", "YES (owns it)", "YES (public)", "YES (issued it)"],
    ["m = H(identity_data)", "YES", "YES (computes)", "NO", "YES"],
    ["M = m*G", "YES", "YES (decrypts)", "NO", "NO"],
    ["E_alice (registered)", "YES", "sees on-chain", "sees on-chain", "NO"],
    ["E_bob (re-encrypted)", "YES (created)", "YES (decrypts)", "sees on-chain", "NO"],
```

```

["sk_alice",          "YES",          "NO",          "NO", "NO"],
["sk_bob",           "NO",           "YES",         "NO", "NO"],
["PS signature sigma", "YES",          "NO",          "NO", "YES (created)"],
["sigma' (rerandomized)", "YES",          "sees on-chain", "sees on-chain", "UNLINKABLE"],
["Chaum-Pedersen proof", "YES (created)", "verified on-chain", "sees on-chain", "N/A"],
["Full receipt",      "YES",          "YES",         "NO", "NO"],
["Link sigma to sigma'", "NO (stat. unlinkable)", "NO", "NO", "NO"],
]

```

Artifact	Alice	Bob	Eve	Issuer
identity_data (Alice)	YES (owns it)	YES (Holochain)	NO	YES (issued it)
identity_data (Bob)	YES (public)	YES (owns it)	YES (public)	YES (issued it)
$m = H(\text{identity_data})$	YES	YES (computes)	NO	YES
$M = m * G$	YES	YES (decrypts)	NO	NO
E_alice (registered)	YES	sees on-chain	sees on-chain	NO
E_bob (re-encrypted)	YES (created)	YES (decrypts)	sees on-chain	NO
sk_alice	YES	NO	NO	NO
sk_bob	NO	YES	NO	NO
PS signature sigma	YES	NO	NO	YES (created)
sigma' (rerandomized)	YES	sees on-chain	sees on-chain	UNLINKABLE
Chaum-Pedersen proof	YES (created)	verified on-chain	sees on-chain	N/A
Full receipt	YES	YES	NO	NO
Link sigma to sigma'	NO (stat. unlinkable)	NO	NO	NO

12 Appendix: The Identity Fountain – Unlinkability Demonstration

Alice derives a second credential from the same PS signature. We verify that no observer can link the two credentials.

```

# === Alice creates a SECOND account (e.g., "savings") ===
# Same m, same sigma, but fresh rerandomization and fresh keys

t_rerand_2 = rand()
sigma_p2_1 = mul(sigma[0], t_rerand_2)
sigma_p2_2 = mul(sigma[1], t_rerand_2)
sigma_p2 = (sigma_p2_1, sigma_p2_2)

sk_alice_2 = rand()
pk_alice_2 = mul(G1, sk_alice_2)

r_enc_2 = rand()
R_alice_2 = mul(G1, r_enc_2)
C_alice_2 = add(M_alice, mul(pk_alice_2, r_enc_2))
E_alice_2 = (R_alice_2, C_alice_2)

# Verify: sigma_p2 is a valid PS signature on the same m
lhs2 = pairing(add(IPK_X, mul(IPK_Y, m_alice)), sigma_p2[0])
rhs2 = pairing(G2, sigma_p2[1])
assert lhs2 == rhs2, "Second credential PS check FAILED"

```

```

print("Second credential derived (same identity, fresh account):")
print(f"  pk_alice_2 = {pt(pk_alice_2)}")
print(f"  E_alice_2  = (R, C) -- independent ciphertext")
print(f"  sigma'_2   = {pt(sigma_p2[0])}")
print()
print("Unlinkability check:")
print(f"  sigma'_1 (acct 1) = {pt(sigma_p[0])}")
print(f"  sigma'_1 (acct 2) = {pt(sigma_p2[0])}")
print(f"  pk (acct 1)      = {pt(pk_alice)}")
print(f"  pk (acct 2)      = {pt(pk_alice_2)}")
print(f"  E.R (acct 1)     = {pt(R_alice)}")
print(f"  E.R (acct 2)     = {pt(R_alice_2)}")
print()
print("  All values are independent.  No shared structure.")
print("  PS rerandomization is STATISTICALLY unlinkable --")
print("  not merely computationally hard, but information-")
print("  theoretically impossible to correlate.")
print()
print("  Both credentials verify under the same issuer public key,")
print("  but zero bits of mutual information exist between them.")

```

```

Second credential derived (same identity, fresh account):
  pk_alice_2 = (249488..., 787692...)
  E_alice_2  = (R, C) -- independent ciphertext
  sigma'_2   = ( 2040..., 929250...)

```

```

Unlinkability check:
  sigma'_1 (acct 1) = (469693..., 489539...)
  sigma'_1 (acct 2) = ( 2040..., 929250...)
  pk (acct 1)      = (199337..., 148784...)
  pk (acct 2)      = (249488..., 787692...)
  E.R (acct 1)     = (540780..., 767503...)
  E.R (acct 2)     = (738077..., 584462...)

```

```

All values are independent.  No shared structure.
PS rerandomization is STATISTICALLY unlinkable --
not merely computationally hard, but information-
theoretically impossible to correlate.

```

```

Both credentials verify under the same issuer public key,
but zero bits of mutual information exist between them.

```